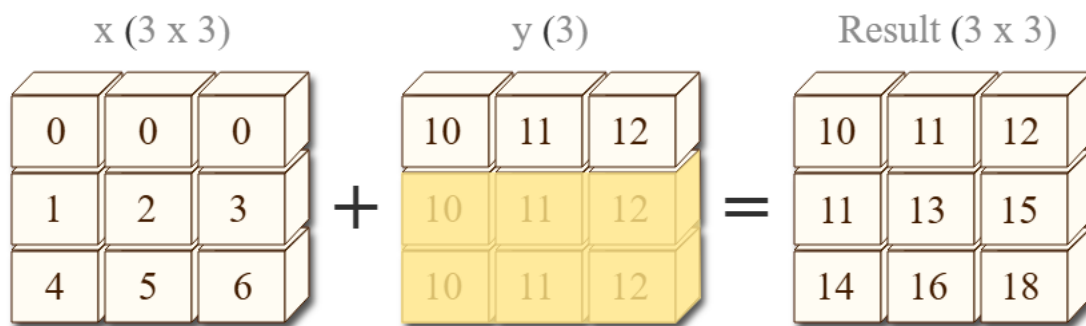


Broadcasting

The term broadcasting refers to the ability of NumPy to treat arrays of different shapes during arithmetic operations. Arithmetic operations on arrays are usually done on corresponding elements. If two arrays are of exactly the same shape, then these operations are smoothly performed.



```
import numpy as np
a = np.array([1,2,3,4])
b = np.array([10,20,30,40])
c = a * b
print(c)
```

output:

```
[ 10  40  90 160]
```

If the dimensions of two arrays are dissimilar, element-to-element operations are not possible. However, operations on arrays of non-similar shapes is still possible in NumPy, because of the broadcasting capability. The smaller array is broadcast to the size of the larger array so that they have compatible shapes.

Size in each dimension of the output shape is maximum of the input sizes in that dimension. An input can be used in calculation, if its size in a particular dimension matches the output size or its value is exactly 1.

If an input has a dimension size of 1, the first data entry in that dimension is used for all calculations along that dimension.

```
import numpy as np
a = np.array([[0.0, 0.0, 0.0], [10.0, 10.0, 10.0], [20.0,
20.0, 20.0], [30.0, 30.0, 30.0]])
b = np.array([1.0, 2.0, 3.0])
print('First array:')
print(a)

print('Second array:')
print(b)

print('First Array + Second Array')
print(a + b)
```

output:

First array:

```
[[ 0.  0.  0.]
 [10. 10. 10.]
 [20. 20. 20.]
 [30. 30. 30.]]
```

Second array:

```
[1. 2. 3.]
```

First Array + Second Array

```
[[ 1.  2.  3.]
```

```
 [11. 12. 13.]
```

```
 [21. 22. 23.]
```

```
 [31. 32. 33.]]
```

Mathematical Functions

Quite understandably, NumPy contains a large number of various mathematical operations. NumPy provides standard trigonometric functions, functions for arithmetic operations, handling complex numbers, etc.

Trigonometric Functions

NumPy has standard trigonometric functions which return trigonometric ratios for a given angle in radians.

```
import numpy as np
```

```
a = np.array([0,30,45,60,90])
```

```
# Convert to radians by multiplying with pi/180
```

```
print('Sine of different angles:')
```

```
print(np.sin(a * np.pi / 180))
```

```
print('\n')
```

```
print('Cosine values for angles in array:')
```

```
print(np.cos(a * np.pi / 180))
```

```
print('\n')
```

```
print('Tangent values for given angles:')
```

```
print(np.tan(a * np.pi / 180))
```

output:

Sine of different angles:

```
[0.          0.5          0.70710678 0.8660254  1.          ]
```

Cosine values for angles in array:

```
[1.00000000e+00 8.66025404e-01 7.07106781e-01 5.00000000e-01  
6.12323400e-17]
```

Tangent values for given angles:

```
[0.00000000e+00 5.77350269e-01 1.00000000e+00 1.73205081e+00  
1.63312394e+16]
```

arcsin, arcos, and arctan functions return the trigonometric inverse of sin, cos, and tan of the given angle. The result of these functions can be verified by numpy.degrees() function by converting radians to degrees.

```
import numpy as np
```

```
a = np.array([0,30,45,60,90])
```

```
print('Array containing sine values:')
```

```
sin = np.sin(a*np.pi/180)
```

```
print(sin)
```

```
print('Compute sine inverse of angles. Returned values are in  
radians.')
```

```
inv = np.arcsin(sin)
```

```
print(inv)
```

```
print('\n')
```

```
print('Check result by converting to degrees:')
```

```
print(np.degrees(inv))
```

```
print('\n')
```

```
print('arccos and arctan functions behave similarly:')
```

```
cos = np.cos(a*np.pi/180)
```

```
print(cos)
```

```
print('\n')
```

```
print('Inverse of cos:')
```

```
inv = np.arccos(cos)
```

```
print(inv)
```

```
print('\n')
```

```
print('In degrees:')
```

```
print(np.degrees(inv))
```

```
print('\n')
```

```
print('Tan function:')
tan = np.tan(a*np.pi/180)
print(tan)
print('\n')
```

```
print('Inverse of tan:')
inv = np.arctan(tan)
print(inv)
print('\n')
```

```
print('In degrees:')
print(np.degrees(inv))
```

output:

Array containing sine values:

```
[0.          0.5          0.70710678 0.8660254  1.          ]
```

Compute sine inverse of angles. Returned values are in radians.

```
[0.          0.52359878 0.78539816 1.04719755 1.57079633]
```

Check result by converting to degrees:

```
[ 0. 30. 45. 60. 90.]
```

arccos and arctan functions behave similarly:

```
[1.00000000e+00 8.66025404e-01 7.07106781e-01 5.00000000e-01
 6.12323400e-17]
```

Inverse of cos:

[0. 0.52359878 0.78539816 1.04719755 1.57079633]

In degrees:

[0. 30. 45. 60. 90.]

Tan function:

[0.00000000e+00 5.77350269e-01 1.00000000e+00 1.73205081e+00
1.63312394e+16]

Inverse of tan:

[0. 0.52359878 0.78539816 1.04719755 1.57079633]

In degrees:

[0. 30. 45. 60. 90.]

Functions for Rounding

1. `numpy.around()`

This is a function that returns the value rounded to the desired precision.

The function takes the following parameters.

Syntax:

`numpy.around(a, decimals)`

- `a` : Input data

- decimals : The number of decimals to round to. Default is 0. If negative, the integer is rounded to position to the left of the decimal point

```
import numpy as np
a = np.array([1.0, 5.55, 123, 0.567, 25.532])
print('Original array:')
print(a)
print('\n')

print('After rounding:')
print(np.around(a))
print(np.around(a, decimals=1))
```

output:

Original array:

```
[ 1.      5.55 123.      0.567 25.532]
```

After rounding:

```
[ 1.    6. 123.    1.   26.]
```

```
[ 1.    5.6 123.    0.6 25.5]
```

2. numpy.ceil()

The ceil() function returns the ceiling of an input value, i.e. the ceil of the scalar x is the smallest integer i, such that $i \geq x$.

```
import numpy as np
a = np.array([-1.7, 1.5, -0.2, 0.6, 10])
print('The given array:')
print(a)
print('\n')
print('The modified array:')
print(np.ceil(a))
```

output:

The given array:

```
[-1.7  1.5 -0.2  0.6 10. ]
```

The modified array:

```
[-1.  2. -0.  1. 10.]
```

3. `numpy.floor()`

This function returns the largest integer not greater than the input parameter.

The floor of the scalar x is the largest integer i , such that

$i \leq x$. Note that in Python, flooring always is rounded away from 0.

```
import numpy as np
a = np.array([-1.7, 1.5, -0.2, 0.6, 10])
print('The given array:')
print(a)
print('\n')
```

```
print('The modified array:')  
print(np.floor(a))
```

output:

The given array:

```
[-1.7  1.5 -0.2  0.6 10. ]
```

The modified array:

```
[-2.  1. -1.  0. 10.]
```

Arithmetic Operations

Input arrays for performing arithmetic operations such as `add()`, `subtract()`, `multiply()` and `divide()` must be either of the same shape or should conform to array broadcasting rules.

```
import numpy as np  
a = np.array([[10,20,30],[40,50,60],[70,80,90]])  
print(a)  
print('\n')  
  
print('Second array:')  
b = np.array([10,10,10])  
print(b)  
print('\n')  
  
print('Add the two arrays:')
```

```
print(np.add(a, b))
print('\n')

print('Subtract the two arrays:')
print(np.subtract(a, b))
print('\n')

print('Multiply the two arrays:')
print(np.multiply(a, b))
print('\n')

print('Divide the two arrays:')
print(np.divide(a, b))
```

output:

```
[[10 20 30]
 [40 50 60]
 [70 80 90]]
```

Second array:

```
[10 10 10]
```

Add the two arrays:

```
[[ 20  30  40]
 [ 50  60  70]
 [ 80  90 100]]
```

Subtract the two arrays:

```
[[ 0 10 20]
 [30 40 50]
 [60 70 80]]
```

Multiply the two arrays:

```
[[100 200 300]
 [400 500 600]
 [700 800 900]]
```

...

Divide the two arrays:

```
[[1. 2. 3.]
 [4. 5. 6.]
 [7. 8. 9.]]
```

4. `numpy.reciprocal()`

This function returns the reciprocal of argument, element-wise. For elements with absolute values larger than 1, the result is always 0 because of the way in which Python handles integer division. For integer 0, an overflow warning is issued.

```
import numpy as np
a = np.array([0.25, 1.33, 1, 0, 100])
print('Our array is:')
print(a)
print('\n')
print('After applying reciprocal function:')
print(np.reciprocal(a))
print('\n')
```

output

Our array is:

```
[ 0.25  1.33  1.    0.   100. ]
```

After applying reciprocal function:

```
[4.          0.7518797 1.          inf 0.01      ]
```

5. numpy.power()

This function treats elements in the first input array as base and returns it raised to the power of the corresponding element in the second input array.

```
import numpy as np
a = np.array([10,100,1000])
print('Our array is:')
print(a)
print('\n')
print('Applying power function:')
print(np.power(a, 2))
```

```
print('\n')

print('Second array:')
b = np.array([1,2,3])
print(b)
print('\n')

print('Applying power function again:')
print(np.power(a, b))
```

output:

Our array is:

```
[ 10  100 1000]
```

Applying power function:

```
[    100   10000 1000000]
```

Second array:

```
[1 2 3]
```

Applying power function again:

```
[          10       10000 10000000000]
```

6. `numpy.mod()`

This function returns the remainder of division of the corresponding

elements in the input array. The function `numpy.remainder()` also produces the same result.

```
import numpy as np
a = np.array([10,20,30])
b = np.array([3,5,7])

print('First array:')
print(a)
print('\n')

print('Second array:')
print(b)
print('\n')

print('Applying mod() function:')
print(np.mod(a, b))
print('\n')

print('Applying remainder() function:')
print(np.remainder(a, b))
```

output:

First array:

[10 20 30]

Second array:

[3 5 7]

Applying mod() function:

[1 0 2]

Applying remainder() function:

[1 0 2]

Iterating Over Array

NumPy package contains an iterator object `numpy.nditer`. It is an efficient multidimensional iterator object using which it is possible to iterate over an array. Each element of an array is visited using Python's standard Iterator interface.

```
import numpy as np
a = np.array([[0,5,10,15],[20,25,30,35],[40,45,50,55]])
print('Original array is:')
print(a)

print('Modified array is:')
for x in np.nditer(a):
    print(x, end=' ')
```

output:

Original array is:

```
[[ 0  5 10 15]
 [20 25 30 35]
 [40 45 50 55]]
```

Modified array is:

```
0 5 10 15 20 25 30 35 40 45 50 55
```

Iteration Order

If the same elements are stored using F-style order, the iterator chooses the more efficient way of iterating over an array.

```
import numpy as np
a = np.array([[0,5,10,15],[20,25,30,35],[40,45,50,55]])
print('Original array is:')
print(a)

print('Sorted in C-style order:')
c = a.copy(order = 'C')
print(c)
for x in np.nditer(c):
    print(x, end=' ')

print('\n Sorted in F-style order:')
c = a.copy(order = 'F')
print(c)
for x in np.nditer(c):
```

```
print(x, end=' ')
```

output:

Original array is:

```
[[ 0  5 10 15]
 [20 25 30 35]
 [40 45 50 55]]
```

Sorted in C-style order:

```
[[ 0  5 10 15]
 [20 25 30 35]
 [40 45 50 55]]
```

0 5 10 15 20 25 30 35 40 45 50 55

Sorted in F-style order:

```
[[ 0  5 10 15]
 [20 25 30 35]
 [40 45 50 55]]
```

0 20 40 5 25 45 10 30 50 15 35 55

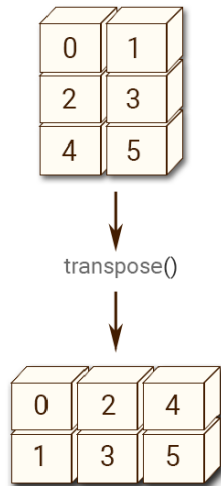
Transpose Operations

1. `numpy.transpose`

This function permutes the dimension of the given array. It returns a view wherever possible. The function takes the following parameters.

Syntax:

```
numpy.transpose(arr)
```



```
import numpy as np
a = np.arange(12).reshape(3,4)
print('The original array is:')
print(a)

print('The transposed array is:')
print(np.transpose(a))
```

output:

The original array is:

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

The transposed array is:

```
[[ 0  4  8]
 [ 1  5  9]
 [ 2  6 10]]
```

```
[ 3  7 11]]
```

2. numpy.ndarray.T

This function belongs to ndarray class. It behaves similar to numpy.transpose.

```
import numpy as np
a = np.arange(12).reshape(3,4)
print('The original array is:')
print(a)

print('Array after applying the function:')
print(a.T)
```

output:

The original array is:

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

Array after applying the function:

```
[[ 0  4  8]
 [ 1  5  9]
 [ 2  6 10]
 [ 3  7 11]]
```